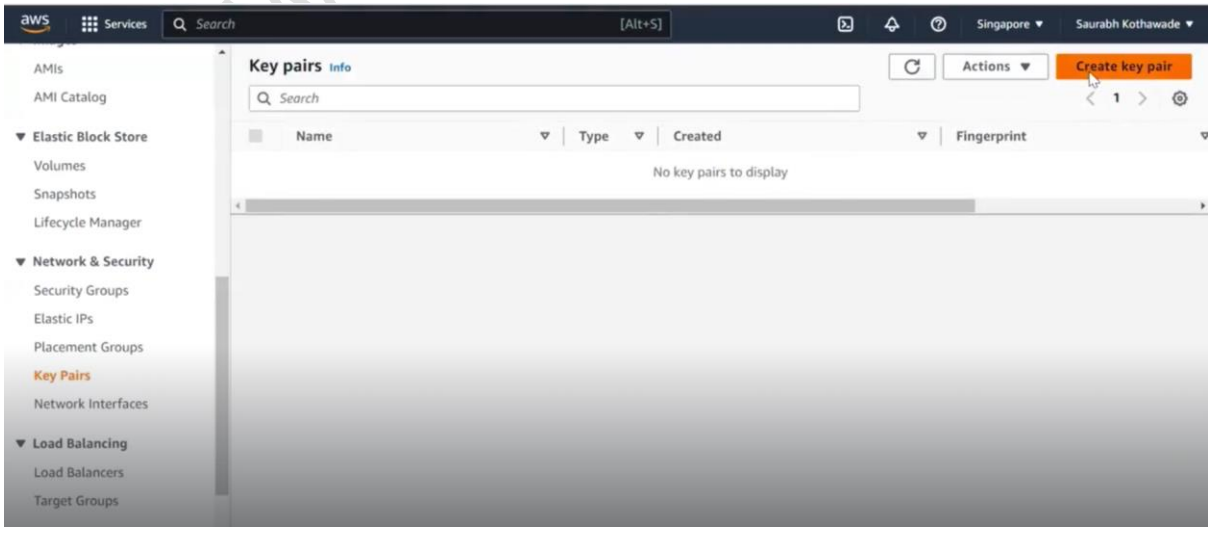
EKS Session

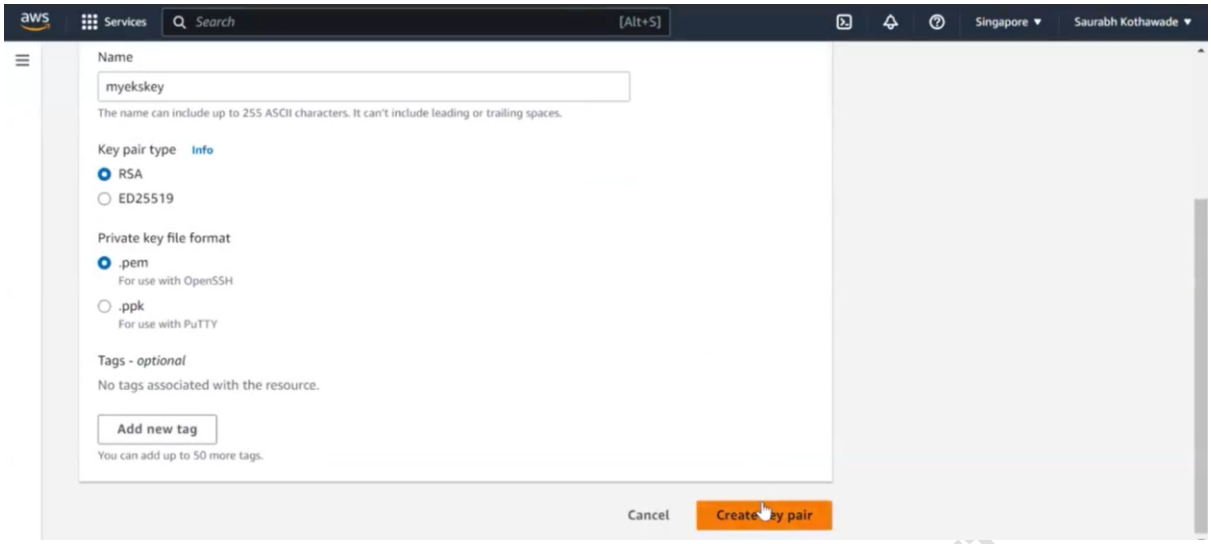
Summary 29-04-2023

- **To launch a Kubernetes cluster using AWS EKS:**

```
Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~
$ eksctl create  cluster \
--name  mycluster \
--region ap-southeast-1 \
--version 1.25 \
--with-oidc \
--nodegroup-name myng1 \
--nodes 3 \
--nodes-min 3 \
--nodes-max 5 \
--ssh-access \
--ssh-public-key myekskey \
--enable-ssm \
--node-private-networking \
--managed \
--instance-types  t3.medium \
--asg-access \
--external-dns-access \
--full-ecr-access \
--appmesh-access \
--alb-ingress-access \
```

- We have to create a public key because we need to provide it when creating the Kubernetes cluster.





- After the EKS cluster has been launched successfully, get the cluster:

```
$ eksctl get cluster --region ap-southeast-1
NAME            REGION          EKSCTL CREATED
mycluster       ap-southeast-1  True
```

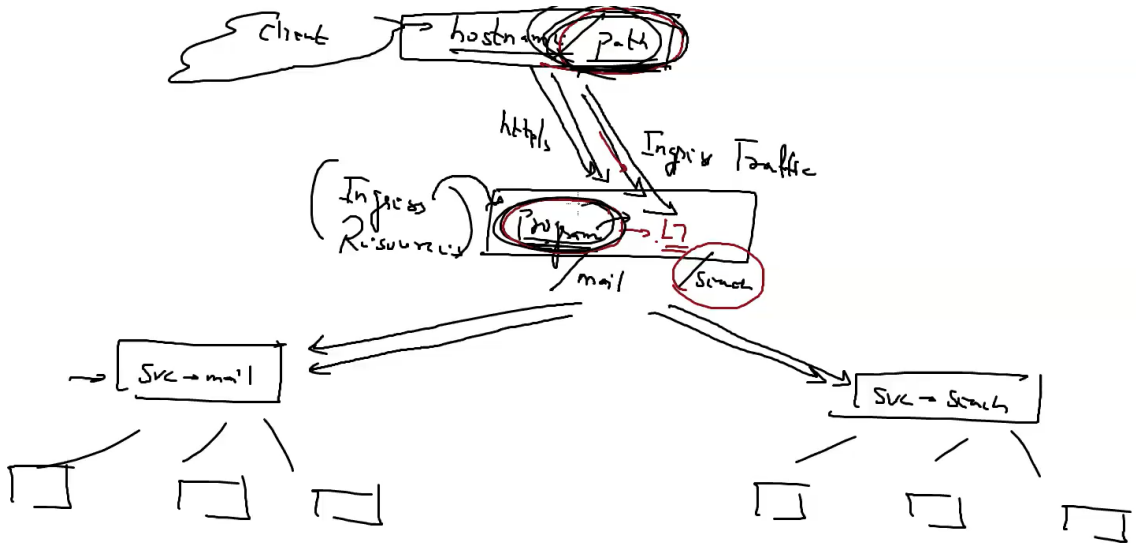
- To get all the nodes:

```
$ kubectl get nodes
NAME                                              STATUS   ROLES    AGE   VERS
ION
ip-192-168-31-251.ap-southeast-1.compute.internal   Ready    <none>   12m   v1.2
5.7-eks-a59e1f0
ip-192-168-60-33.ap-southeast-1.compute.internal    Ready    <none>   27m   v1.2
5.7-eks-a59e1f0
ip-192-168-73-37.ap-southeast-1.compute.internal    Ready    <none>   12m   v1.2
5.7-eks-a59e1f0
```

- **Path-based routing** allows you to route traffic to different services based on the path in the incoming request URL.

- **Ingress traffic** is the incoming traffic that is routed to a particular service. An **Ingress resource** acts as a gateway to your cluster and can be used to route traffic to different services based on various criteria, such as the hostname, URL path, or request method.

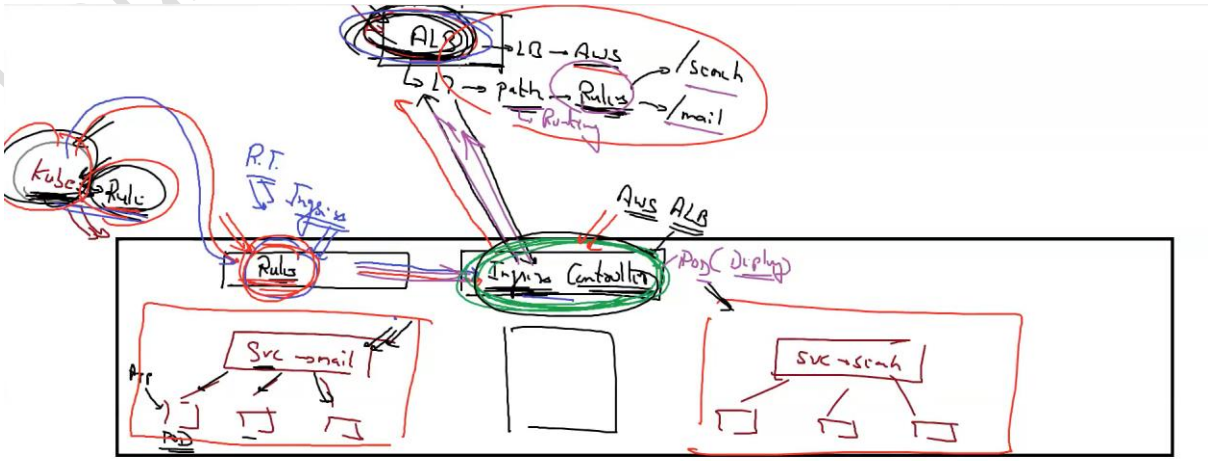
- To handle incoming traffic through an Ingress resource, you need to have an **Ingress controller** running in your Kubernetes cluster. The Ingress controller is responsible for implementing the rules defined in the Ingress resource and routing traffic to the appropriate backend services.



- The **ALB Ingress Controller** is one of several options for implementing an Ingress controller in Kubernetes. It provides an easy way to route traffic to services running in a Kubernetes cluster using an AWS Application Load Balancer, which can scale to handle high-traffic loads and provide path-based routing.

- **Ingress** is a Kubernetes resource that defines rules for routing external traffic to internal services in a Kubernetes cluster.

- So, while Ingress manages the rules for routing external traffic to internal services in a Kubernetes cluster, the AWS ALB Ingress Controller is responsible for translating these rules into AWS-specific load balancing rules and configuring the load balancer accordingly.



- The **"--alb-ingress-access"** option is used to enable the AWS ALB Ingress Controller and connect it to the cluster.

- This option creates an IAM policy and role that grants the necessary permissions to the ALB Ingress Controller to interact with the AWS resources required for load balancing, and configures the ALB Ingress Controller to use the AWS Application Load Balancer to route traffic to services in the cluster based on the rules defined in the Ingress resource.

- **To deploy the AWS Load Balancer (ALB) Controller to an Amazon EKS cluster:**

- **Create an IAM policy**.

- Download an IAM policy for the AWS Load Balancer Controller that allows it to make calls to AWS APIs on your behalf.

```
$ curl -O https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-co
ntroller/v2.4.7/docs/install/iam_policy.json
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  8386  100  8386    0     0  15189      0 --:--:-- --:--:-- --:--:-- 15192
```

- Create an IAM policy using the policy downloaded in the previous step.

```
$ aws iam create-policy     --policy-name AWSLoadBalancerControllerIAMPolicy
 --policy-document file://iam_policy.json
{
    "Policy": {
        "PolicyName": "AWSLoadBalancerControllerIAMPolicy",
        "PolicyId": "ANPA42QIP75SQHSDCPWU7",
        "Arn": "arn:aws:iam::881559863141:policy/AWSLoadBalancerControllerIAMPol
icy",
        "Path": "/",
        "DefaultVersionId": "v1",
        "AttachmentCount": 0,
        "PermissionsBoundaryUsageCount": 0,
        "IsAttachable": true,
        "CreateDate": "2023-04-29T06:32:00+00:00",
        "UpdateDate": "2023-04-29T06:32:00+00:00"
    }
}
```

- Attach the above policy to the EKS.

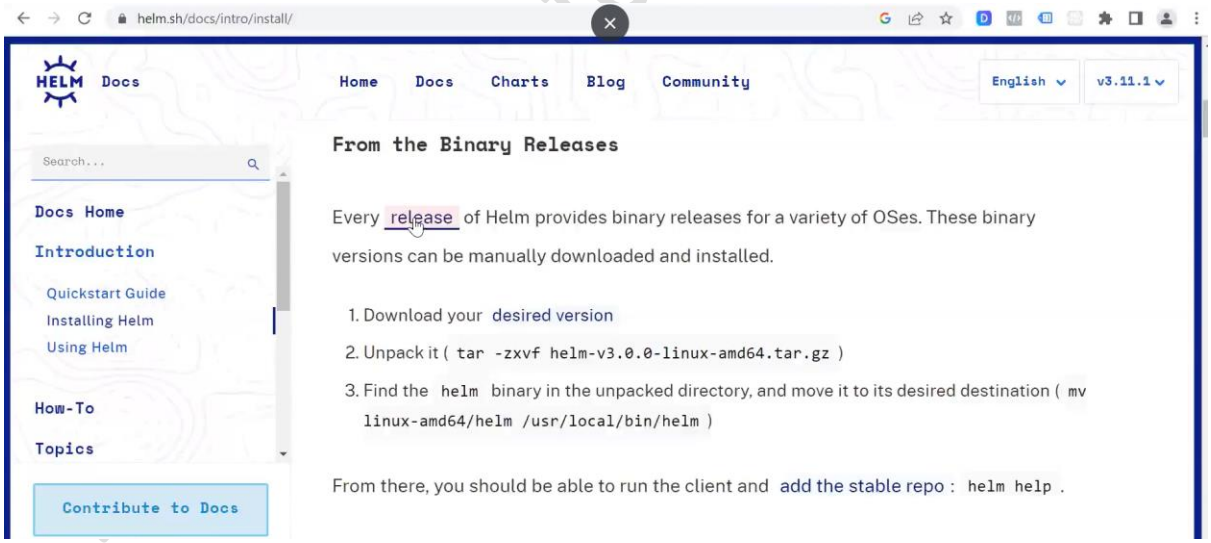
```
$ eksctl create iamserviceaccount \
  --cluster=mycluster \
--region ap-southeast-1 \
  --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn=arn:aws:iam::881559863141:policy/AWSLoadBalancerController
IAMPolicy \
  --approve
```

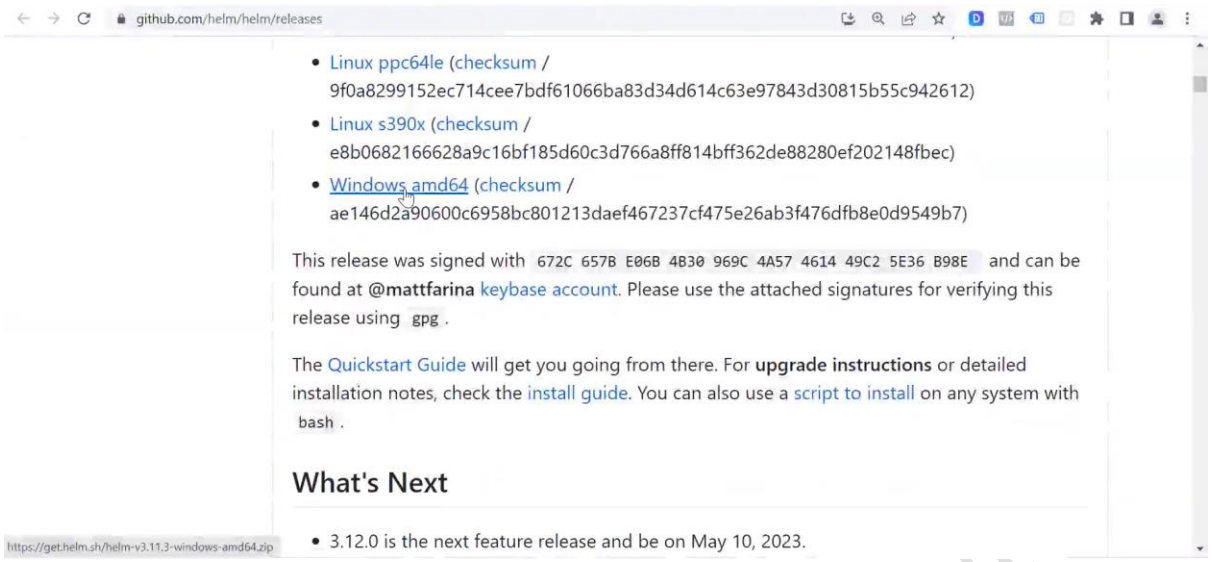
```
$ kubectl get  sa aws-load-balancer-controller  -n kube-system
NAME                           SECRETS   AGE
aws-load-balancer-controller   0         82s
```

```
$ kubectl describe   sa aws-load-balancer-controller  -n kube-system
Name:                aws-load-balancer-controller
Namespace:           kube-system
Labels:              app.kubernetes.io/managed-by=eksctl
Annotations:         eks.amazonaws.com/role-arn: arn:aws:iam::881559863141:role/
AmazonEKSLoadBalancerControllerRole
Image pull secrets:  <none>
Mountable secrets:   <none>
Tokens:              <none>
Events:              <none>
```

- To enable authentication between the Kubernetes Ingress Controller and the AWS ALB, you can use **OIDC**. For this "**--with-oidc**" is used.

- Install Helm on your local machine






```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe version
version.BuildInfo{Version:"v3.11.3", GitCommit:"323249351482b3bbfc9f5004f65d400aa70f9ae7", Gi
tTreeState:"clean", GoVersion:"go1.20.3"}
```

- Add the eks-charts repository and update your local repo to make sure that you have the most recent charts.

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe  repo add eks https://aws.githu
b.io/eks-charts
"eks" already exists with the same configuration, skipping

C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. ⎈Happy Helming!⎈
```

- Install the AWS Load Balancer Controller chart using the helm install command:

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>helm.exe  install aws-load-balancer-cont
roller eks/aws-load-balancer-controller -n kube-system  --set clusterName=mycluster    --set serviceAccount.cre
ate=false   --set serviceAccount.name=aws-load-balancer-controller
NAME: aws-load-balancer-controller
LAST DEPLOYED: Sat Apr 29 12:23:30 2023
NAMESPACE: kube-system
STATUS: deployed
REVISION:
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
```

```
C:\Users\Vimal Daga\Downloads\helm-v3.11.3-windows-amd64\windows-amd64>kubectl get pods  -n kube-system
NAME                                           READY   STATUS    RESTARTS   AGE
aws-load-balancer-controller-57cc94c574-9qv9t  1/1     Running   0          22s
aws-load-balancer-controller-57cc94c574-x5xgl  1/1     Running   0          22s
aws-node-k9prj                                 1/1     Running   0          43m
aws-node-lwbsc                                 1/1     Running   0          44m
aws-node-t669x                                 1/1     Running   0          59m
coredns-7cc96f45bb-jq4t7                       1/1     Running   0          68m
coredns-7cc96f45bb-nvcdv                       1/1     Running   0          68m
kube-proxy-7fm7s                               1/1     Running   0          59m
kube-proxy-ctmrw                               1/1     Running   0          44m
kube-proxy-mdwxq                               1/1     Running   0          43m
```

- **IngressClass** is used to select the appropriate Ingress controller.

vim ingressclass.yml

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: awesome-class
  annotations:
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: ingress.k8s.aws/alb
~
~
~
~
~
```

```
Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~
$ kubectl apply -f ingressclass.yml
```

```
Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~
$ kubectl get ingressclass
NAME            CONTROLLER            PARAMETERS   AGE
alb             ingress.k8s.aws/alb   <none>       10m
awesome-class   ingress.k8s.aws/alb   <none>       12s
```

- **Deploy an application with service.**

vim deploy.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myweb-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
       env: production
    matchExpressions:
       - { key: env,  operator: In,  values: [ production ] }
  template:
    metadata:
     name: myweb-pod
     labels:
       env: production
    spec:
      containers:
      - name: myweb-con
        image: vimal13/apache-webserver-php
~
~
```

```
Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Desktop/kube_code
$ kubectl apply -f deploy.yml
deployment.apps/myweb-deploy created
```

```
$ kubectl get pods --show-labels
NAME                             READY   STATUS    RESTARTS   AGE   LABELS
myweb-deploy-7bc6cbf7dd-bdjxv    1/1     Running   0          51s   env=productio
n,pod-template-hash=7bc6cbf7dd
myweb-deploy-7bc6cbf7dd-fj7vw    1/1     Running   0          51s   env=productio
n,pod-template-hash=7bc6cbf7dd
myweb-deploy-7bc6cbf7dd-xrr7x    1/1     Running   0          51s   env=productio
n,pod-template-hash=7bc6cbf7dd
```

vim service.yml

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  labels:
    app: myweb-service
spec:
  selector:
    env: production
  type: NodePort
  ports:
    - nodePort: 31000
      port: 80
      targetPort: 80
~
~
~
```

```
Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Desktop/kube_code
$ kubectl apply -f service.yml
service/my-service created

Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Desktop/kube_code
$ kubectl get svc
NAME          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes    ClusterIP   10.100.0.1      <none>        443/TCP        87m
my-service    NodePort    10.100.76.202   <none>        80:31000/TCP   5s
```

- **Ingress** is used to define the routing rules for external traffic.
- When you create an Ingress resource, specify the ingressClassName field to associate the resource with a specific IngressClass.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minimal-ingress
  annotations:
    alb.ingress.kubernetes.io/load-balancer-name: "myalb-test"
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/healthcheck-protocol: HTTP
    alb.ingress.kubernetes.io/healthcheck-port: traffic-port
    alb.ingress.kubernetes.io/healthcheck-path: /index.php
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '15'
    alb.ingress.kubernetes.io/healthcheck-timeout-seconds: '5'
    alb.ingress.kubernetes.io/success-codes: '200'
    alb.ingress.kubernetes.io/healthy-threshold-count: '2'
    alb.ingress.kubernetes.io/unhealthy-threshold-count: '2'
spec:
  ingressClassName: awesome-class
  defaultBackend:
    service:
      name: my-service
      port:
        number: 80
```

```
Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Desktop/kube_code
$ kubectl apply -f in.yml
ingress.networking.k8s.io/minimal-ingress created

Vimal Daga@DESKTOP-3E1AGGT MINGW64 ~/Desktop/kube_code
$ kubectl get ingress
NAME              CLASS           HOSTS   ADDRESS
                   PORTS   AGE
minimal-ingress   awesome-class   *       myalb-test-1858659047.ap-southeast-1.e
lb.amazonaws.com   80      4s
```

- Now if we go to load balancer and copy the DNS name and paste it into the browser then we can see the application.